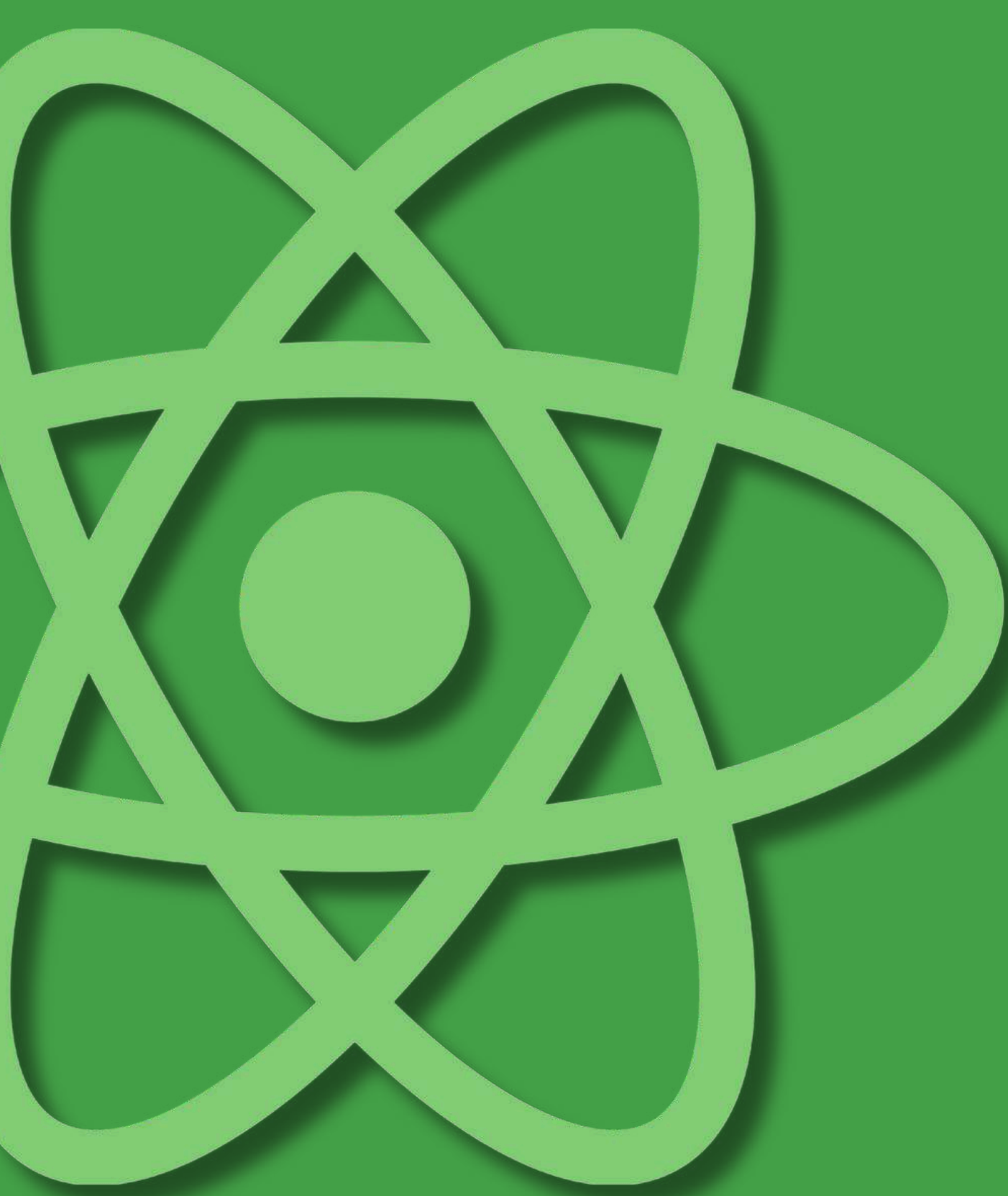




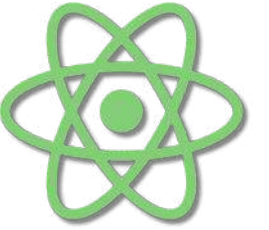
React.js

Digita qui il testo

Docente: Umberto Emanuele

Styling e CSS

CSS Modules



Cosa sono:

- File CSS con scope locale
- Evitano conflitti di nomi
- Importati come oggetti JavaScript



Vantaggi:

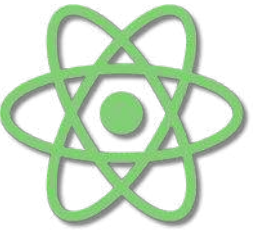
- Class names automaticamente univoci
- Co-location: stili vicino ai componenti
- Tree-shaking: CSS non usato rimosso
- Type-safe con TypeScript



Pattern comuni:

- File `.module.css`
- Import e utilizzo tramite oggetto
- Composizione con `composes`

Styling e CSS



Styled Components e Librerie Moderne

```
const Button = styled.button`
  background: ${props => props
    primary ? "blue",
  } color: white;
  &:hover {
    opacity: 0.8;
  },
};
```

CSS-in-JS:

- Stili scritti in JavaScript/TypeScript
- Props dinamiche per styling condizionale
- Theming integrato



Alternative moderne:

Tailwind CSS con classi utility
CSS Modules + Sass/PostCSS

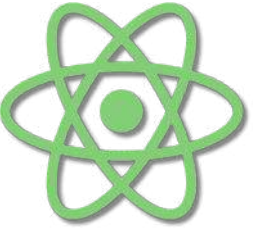
Styled Components:

```
const Button = styled.button`
  background: blue;
  color: white;
  &:hover {
    opacity: 0.8;
  },
};
```

- Template literals per definire stili
- Componenti stilizzati riutilizzabili
- Support per pseudo-selettori e media queries

Styling e CSS

Conditional Style con TypeScript



Styling dinamico:

- Stili che cambiano in base a props o stato
- Classi condizionali
- Inline styles per valori dinamici

Pattern con TypeScript:

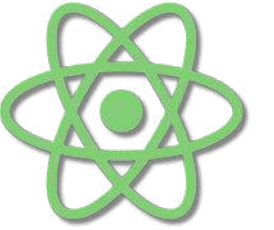
- Type-safe props per variant styling
- Union types per valori permessi
- Utility types per props opzionali

Best practices:

- Evitare inline styles quando possibile
- Librerie helper: `classnames`, `clsx`
- Separare logica di styling da logica business



Styling e CSS



Responsive Design e Media Queries

Approcci:



Media queries in CSS tradizionale



Breakpoint con Styled Components



Hook personalizzati: useMediaQuery

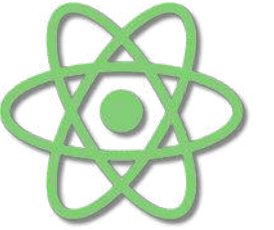


Container queries (CSS moderno)

Mobile-first:

- Partire da layout mobile
- Progressive enhancement per schermi grandi
- Performance su dispositivi mobili

Gestione stato



Gestione Stato useReducer Hook

Quando Usare:

- Stato complesso con logica di update
- Più azioni che modificano lo stato
- Next state dipende dal previous

Reducer pattern:

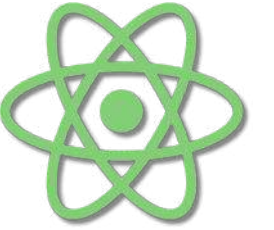
- **Pure function:** (state, action) => newState
- Switch case per azioni diverse
- Type-safe con TypeScript discriminated unions

Vantaggi:

- Logica centralizzata
- Più testabile di useState
- Pattern simile a Redux



Gestione stato



Context API: Condivisione dello Stato

Problema:

- **Props drilling:** passare props attraverso molti livelli
- Stato globale necessario in molti componenti

Context API:

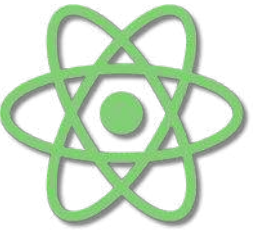
- Condivisione dati senza passare props esplicitamente
- Provider e Consumer pattern
- Hook *useContext* per accesso semplificato

Casi d'uso:

- Tema dell'applicazione
- Autenticazione utente
- Preferenze e impostazioni
- Lingua/localizzazione

Gestione stato

Pattern per Applicazioni Scalabili



Librerie esterne

Separazione delle responsabilità:

- Componenti presentazionali vs container
- Custom hooks per logica riutilizzabile
- Servizi per business logic

State management patterns:

- Lifting state up quando necessario
- State co-location: stato vicino a dove serve
- Composizione su condivisione globale



Zustand: semplice e performante



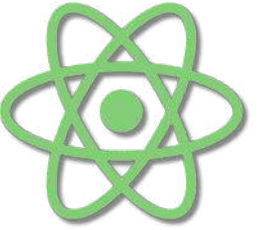
Redux Toolkit: standard per app complesse



Jotai/Recoil: atomic state management

Gestione stato

State Machines e XStat



Concetto:

- Stati finiti ben definiti
- Transizioni esplicite tra stati
- Prevenzione stati impossibili

Vantaggi:

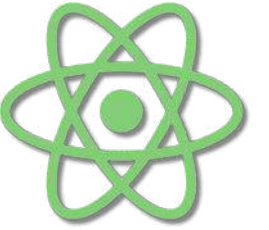
- Logica complessa visualizzabile
- Bug ridotti: stati impossibili esclusi
- Testing deterministico



Casi d'uso:

- Flow multi-step (wizard, checkout)
- Gestione connessione/authenticazione
- UI con molti stati (loading, error, success, idle)

Effetti e Richieste HTTP



useEffect e Ciclo di Vita

Side effects:



Operazioni che escono dal rendering puro



API calls, subscriptions, timers, DOM manipulation

useEffect:



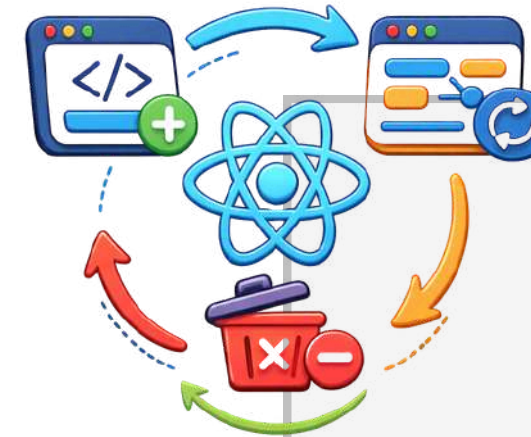
Esegue codice dopo il render



Dependency array per controllare esecuzioni



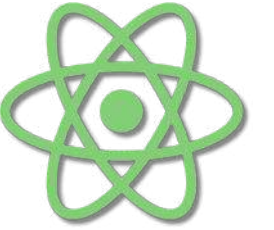
Cleanup function per pulizia



Ciclo di vita:

- **Mount:** primo render
- **Update:** re-render per cambio props/state
- **Unmount:** rimozione componente
- *useEffect* può gestire tutti e tre

Effetti e Richieste HTTP



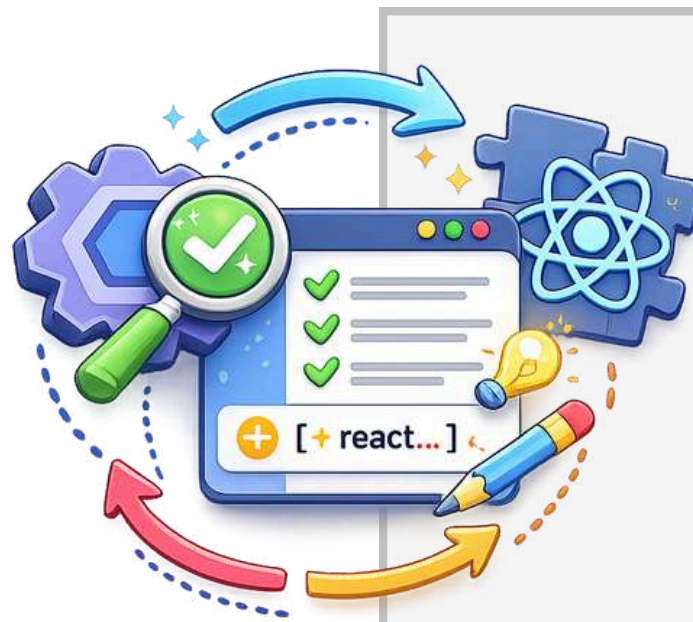
Dependency Array e Rules of Hooks

Dependency array:

- **Array vuoto []**: esegue solo al mount
- **Omesse**: esegue ad ogni render
- **Con dipendenze**: esegue quando cambiano

Rules of Hooks:

- Chiamare hooks solo al top level
- Non in condizioni, loop o funzioni annidate
- Solo in componenti React o custom hooks

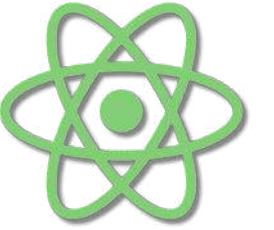


ESLint plugin:

- `eslint-plugin-react-hooks`
- Verifica automatica delle regole
- Suggerimenti per dipendenze mancanti

Effetti e Richieste HTTP

Fetch e Axios per Richieste HTTP



Fetch API:



- Nativa del browser
- Promise-based
- Richiede parsing manuale della risposta

Axios:



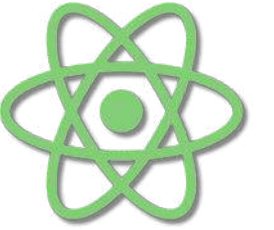
- Libreria HTTP più completa
- Parsing automatico JSON
- Interceptors per richieste/risposte
- Timeout e cancellazione built-in

Gestione errori:

- Try-catch per async/await
- Stati di loading, error, success
- User feedback appropriato



Effetti e Richieste HTTP



TanStack Query (React Query)

Perché TanStack Query:

- Gestione automatica di caching
- Refetching intelligente
- Stati di loading/error gestiti
- Sincronizzazione tra tab/finestre

Concetti chiave:

- **Queries:** fetch di dati
- **Mutations:** modifiche ai dati
- Cache invalidation
- Optimistic updates

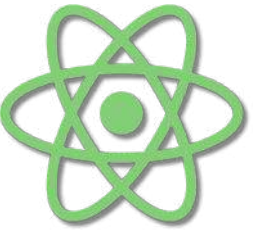
Vantaggi:

- Meno codice boilerplate
- Performance migliorate
- Esperienza utente superiore



Effetti e Richieste HTTP

Gestione Asincrona Avanzata



Pattern async/await:

- Codice più leggibile che promises
- Error handling con try/catch
- Sequential vs parallel requests

Cancellazione richieste:

- AbortController per fetch
- Cleanup in useEffect
- Prevenire memory leaks

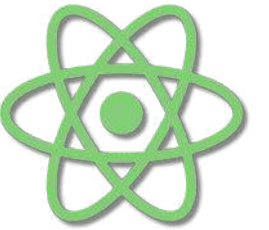
Ottimizzazioni:

- Debouncing di richieste
- Request deduplication
- Prefetching per UX migliore
- Stale-while-revalidate pattern



Routing

React Router v7: Introduzione



Routing:

- Navigazione tra diverse "pagine" in SPA
- URL sincronizzato con UI
- Storia di navigazione (back/forward)

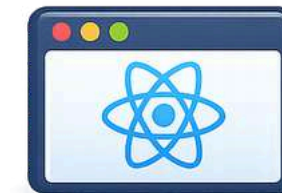
React Router:

- Libreria standard per routing in React
- Dichiarativo e component-based
- Support per nested routes

Installazione e setup



Package react-router-dom



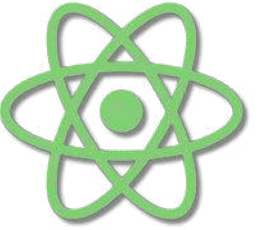
BrowserRouter come wrapper dell'app



Definizione delle routes

Routing

Rotte e Navigazione



Componenti fondamentali:

- `<Routes>`: container delle route
- `<Route>`: definisce path e component
- `<Link>`: navigazione dichiarativa
- `<Navigate>`: redirect programmatico

Navigazione

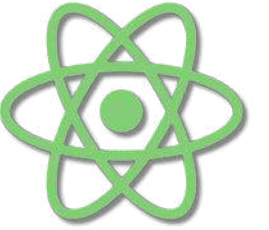
- Link per navigazione utente
- `useNavigate` hook per navigazione programmatica
- Parametri URL e query string

Best practices:

- Route lazy loading per performance
- Protected routes per autenticazione
- 404 page per route non trovate



Routing



Rotte Dinamiche e Parametri

Parametri dinamici:

- Segmenti URL variabili: `/users/:id`
- Accesso tramite `useParams` hook
- Type-safe con TypeScript

Query parameters:

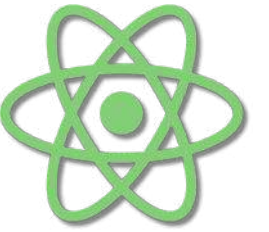
- Parametri opzionali: `/search?q=react`
- Accesso con `useSearchParams` hook
- Utili per filtri, paginazione, ordinamento

Utilizzo:

- Detail pages: `/products/:productId`
- User profiles: `/users/:username`
- Search results con filtri

Routing

Nested Routes



Concetto:

- Route annidate che condividono layout
- Parent route con `<Outlet/>` per children
- Gerarchie di navigation

Esempio tipico:

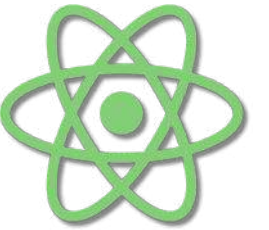
- Dashboard con sidebar
- Settings con tabs Admin
- panel con sezioni

Vantaggi:

- Condivisione di layout e logica
- Navigation persistente (sidebar, header)
- Organizzazione logica dell'app



Routing



Protected Routes e Autenticazione

Protezione route:



Verificare autenticazione prima del render



Redirect a login se non autenticato



Componente wrapper riutilizzabile

Pattern comune:

- HOC (Higher Order Component)
- Route wrapper con check auth
- Context per stato auth globale

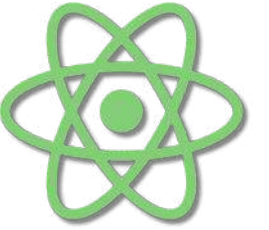
Autorizzazione:



- Ruoli e permessi utente
- Route based on permissions
- Conditional rendering di features

Routing

Code Splitting e Lazy Loading



Problema:

- Bundle JavaScript troppo grande
- Tempo di caricamento iniziale lent



Soluzione:

- `React.lazy()`: import dinamici
- `<Suspense>`: fallback durante caricamento
- Split per route automatico

Vantaggi:

- Caricamento iniziale più veloce
- Codice scaricato on-demand
- Migliore performance percepita

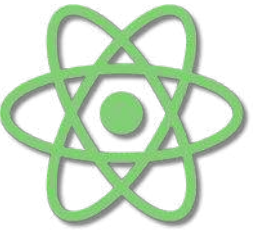


Best practices:

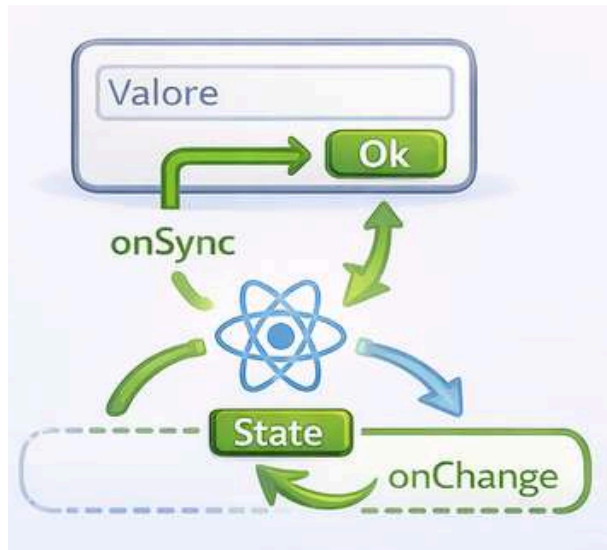
- Lazy load route-level components
- Loading skeleton per UX Error
- boundaries per gestione errori



Form e Validazione



Form Controllati e Non Controllati



Form controllati:

- Valore sincronizzato con stato React
- onChange per ogni modifica
- React è "single source of truth"



Form non controllati:

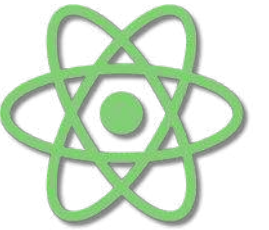
- DOM mantiene lo stato
- Ref per accedere ai valori
- Meno re-render

Quando usare cosa:

- **Controllati:** validazione real-time, logica complessa
- **Non controllati:** form semplici, performance critiche
- **React Hook Form:** best of both worlds

Form e Validazione

React Hook Form



Concetti chiave:

- **register**: registra input nel form
- **handleSubmit**: gestisce submit
- **formState**: errori, validità, stato submit
- **watch**: osserva valori specifici



DX migliorata:

- Meno boilerplate code
- Validazione dichiarativa
- Error handling integrato

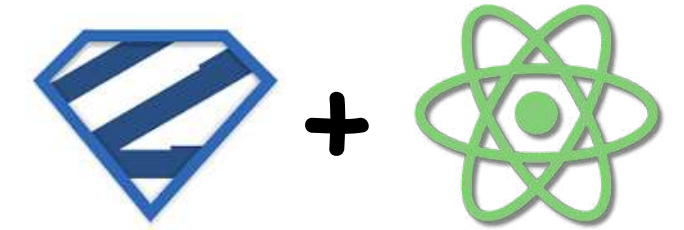
Vantaggi:

- Performance ottimizzate: meno re-render
- API semplice e intuitiva
- Integrazione con librerie di validazione
- Support per TypeScript



Form e Validazione

Validazione con Zod



Zod:



- Schema validation library per TypeScript
- Type inference automatico
- Messaggi di errore personalizzabili

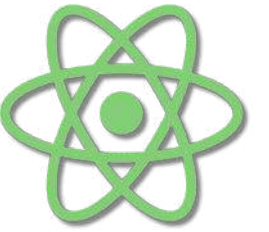
Integrazione con React Hook Form:

- Schema come fonte di validazione
- Type safety end-to-end
- Validazione client e server con stesso schema

Vantaggi:

- Un unico schema per validazione e types
- Composizione di schemi complessi
- Runtime validation type-safe
- Error messages chiari e localizzabili





Form e Validazione

Gestione Errori e UX delle Form

User Experience:

-  Feedback immediato ma non invasivo
-  Messaggi di errore chiari e actionable
-  **Indicatori di stato:** loading, success, error

Accessibilità:

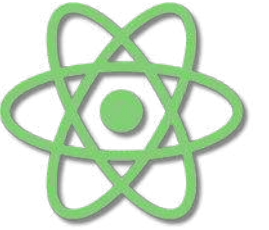
- Label associate agli input
- ARIA attributes per screen readers
- Focus management
- Keyboard navigation

Best practices:

- Validazione on blur o on submit, non on change
- Evidenziare campi con errori
- Disabilitare submit durante invio
- Preservare dati in caso di errore



Performance



Memorization: useMemo e useCallback

UseMemo



Memorizza risultato di computazioni costose



Ricalcola solo quando dipendenze cambiano



Ottimizzazione per calcoli pesanti

Quando usare:

- Calcoli complessi nel render
- Props callback a componenti memoizzati
- Dipendenze di useEffect che sono funzioni

useCallback



Memorizza reference di funzioni



Previene re-creazione inutile



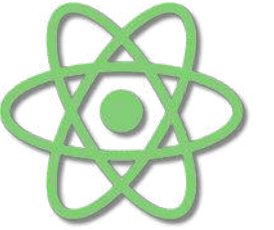
Utile per ottimizzare child components

Attenzione

- Non premature optimization
- Profiling prima di ottimizzare



Performance



React.memo e Component Optimization

React.memo:



HOC per memoizzare componenti



Shallow comparison delle props



Re-render solo se props cambiano

Custom comparison:

- Secondo argomento per logica custom
- Deep comparison quando necessario
- Balance tra complessità e benefici

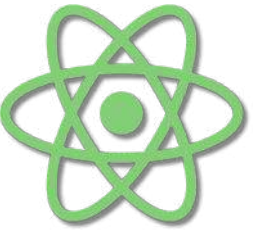


Quando usare:

- Componenti puri e pesanti
- Render frequenti con stesse props
- Liste con molti item

Performance

Virtualizzazione e Large Lists



Problema:

- Render di migliaia di elementi è lento
- DOM sovraccarico



Soluzione: Virtualization

- Render solo elementi visibili
- Recycling di DOM nodes
- Scroll performante

Librerie:



react-window: leggera e veloce



react-virtual: moderna e flessibile

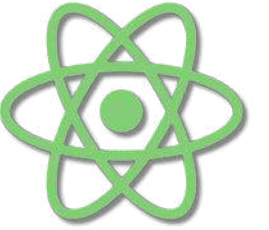


TanStack Virtual: parte dell'ecosistema TanStack

Casi d'uso:

- Liste lunghe
- Tabelle con molte righe
- Chat infiniti

Testing



Introduzione al Testing in React

Tipologie di test:

- Unit test: singoli componenti/funzioni
- Integration test: interazione tra componenti
- E2E test: flussi completi dell'app

Strumenti:



Vitest: test runner moderno e veloce



Jest: standard de facto



React Testing Library: test user-centric



Cypress/Playwright: E2E testing

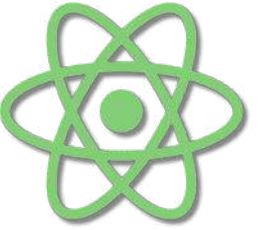
Filosofia:



- Test behavior, non implementation
- **User perspective:** cosa vede l'utente
- Confidence nel refactoring

Testing

React Testing Library



Principi:

- Query per ruolo e label, non classi/id
- Simulare interazioni utente reali
- Async utilities per operazioni asincrone

Query priority:



1. **getByRole**: accessibilità first



2. **getByLabelText**: form inputs



3. **getByText**: contenuto testuale

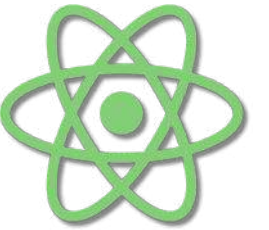


4. **getByTestId**: last resort

User events:

- `userEvent.click(), type(), hover()`
- Simulazione realistica
- Async by default

Deployment



Build e Ottimizzazione per Produzione

Build process:

- `npm run build` con Vite
- Minificazione e bundling
- Tree-shaking automatico
- Asset optimization



Ottimizzazioni:

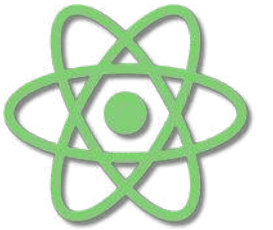
- Code splitting applicato
- Compression (gzip/brotli)
- Cache busting con hash
- Environment variables



Analisi bundle:

- Bundle analyzer per visualizzare dimensioni
- Identificare dipendenze pesanti
- Opportunità di ottimizzazione

Deployment



Deploy su Vercel, Netlify e Alternative

Piattaforme moderne:



Vercel: ottimizzato per React/Next.js



Netlify: CI/CD integrato, form handling



Cloudflare Pages: edge computing



GitHub Pages: hosting gratuito per progetti semplici

Features comuni:

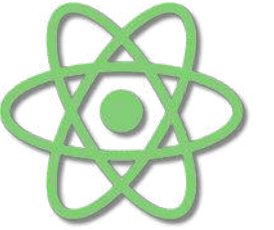
- Deploy automatico da Git
- Preview per PR
- Environment variables
- Custom domains
- HTTPS automatico

Best practices:

- Variabili d'ambiente per config
- Health checks e monitoring
- Rollback strategy



Conclusione



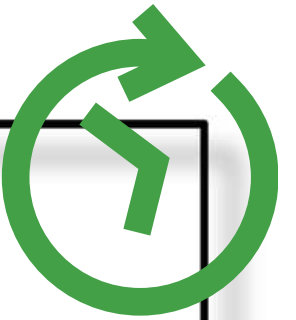
Prossimi Passi nel Vostro Percorso React

Fondamenta acquisite:



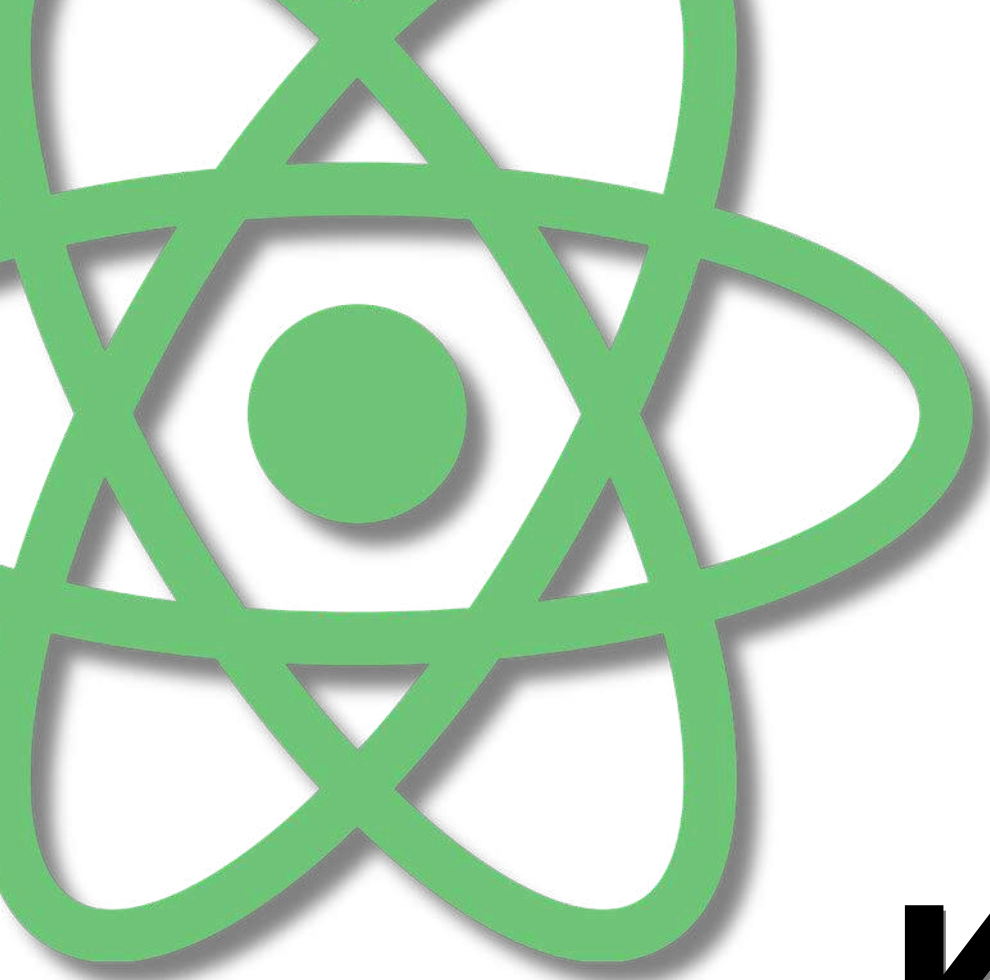
- Setup e tooling moderno
- TypeScript per type safety
- Componenti e composizione
- State management e side effects
- Routing e navigazione
- Form e validazione
- Performance e testing
- Deployment

Approfondimenti futuri:



- Server-side rendering (Next.js, Remix)
- React Server Components
- Animation libraries (Framer Motion)
- State management avanzato
- Design patterns e architetture
- Micro-frontends

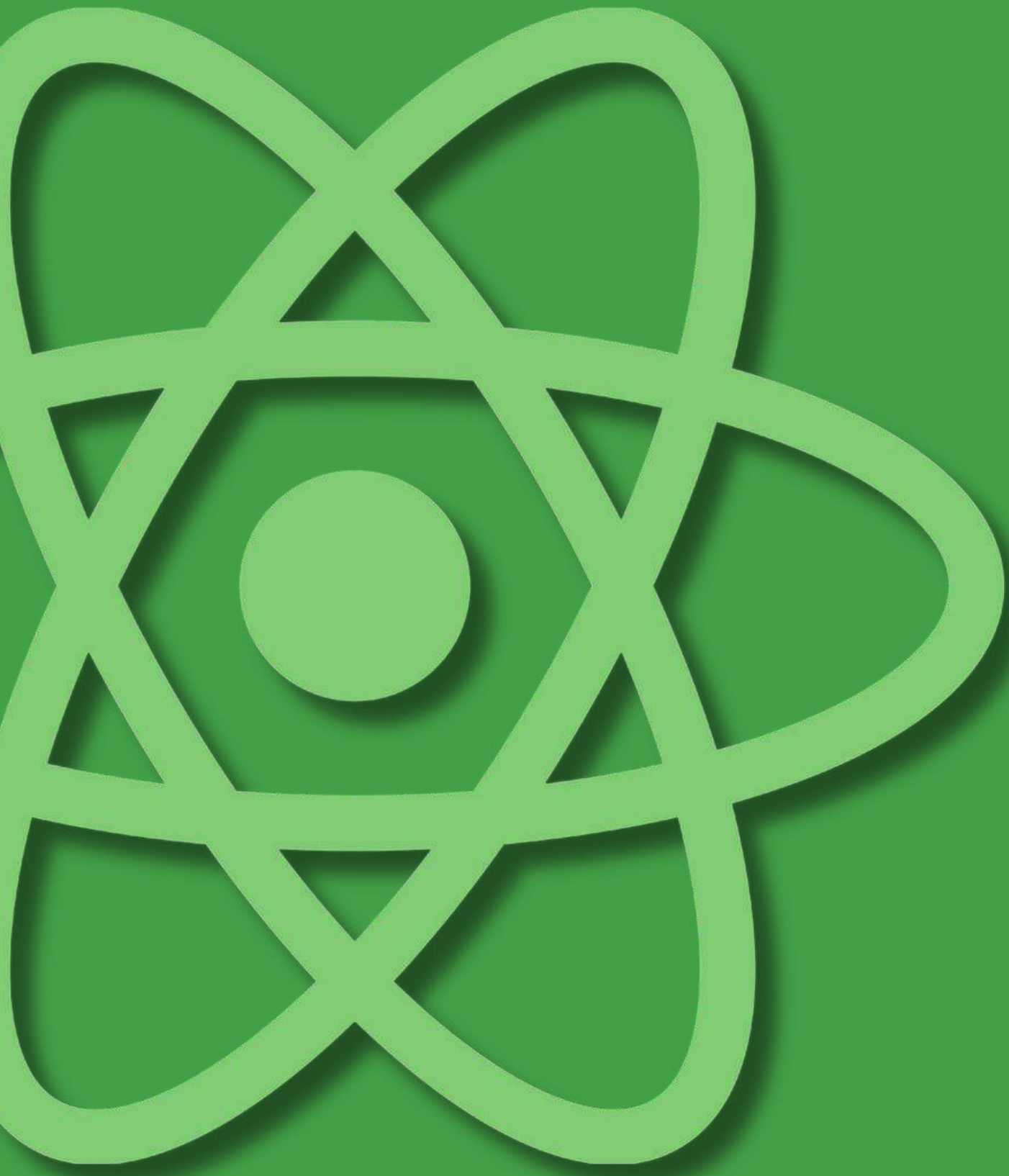




Keep learning.
Keep building.



Labforweb
nerd academy



Grazie